be defined. Each terminal is typically either a variable or a constant, defined on the problem domain.

Example 8.1. Given the following sets of functions and terminals:

$$\mathscr{F} = \{+, -\}, \quad \mathscr{T} = \{x, 1\}$$

a legal GP individual is represented in Figure 8.1.

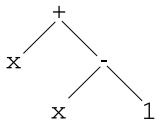

Fig. 8.1 A tree that can be built with the sets $\mathscr{F} = \{+, -\}$ and $\mathscr{T} = \{x, 1\}$

This tree can also be represented by the following LISP-like S-expression (for a definition of LISP S-expressions see, for instance, [Koza, 1992]):

$$(+\ x\ (-\ x\ \ 1))$$

which is the prefix notation representation for expression $x + (x - 1)$.

Closure and Sufficiency of Function Set and Terminal Set. The function and terminal sets should be chosen so as to verify the requirements of *closure* and *sufficiency*. The closure property requires that each of the functions in the function set be able to accept, as its arguments, any value and data type that may possibly be returned by any function in the function set and any value and data type that may possibly be assumed by any terminal in the terminal set. In other words, each function should be well defined for any combination of arguments that it may encounter. The reason why this property must be verified is clearly that programs must be executed in order to assign them a fitness, and a failure in one of the executions of one of the programs composing the population would lead either to a failure of the whole GP system or to the generation of unpredictable results. The function and terminal sets of the previous example clearly satisfy the closure property. The following ones, for instance, don't satisfy this property:

$$\mathscr{F} = \{*, /\}, \quad \mathscr{T} = \{x, 0\}$$

In fact, each evaluation of an expression containing an operation of division by zero would cause an error. In order to use division, but avoid this type of problem, this operation is usually modified in order to verify the closure property (see Section 8.3.1). Respecting the closure property in real-life applications is not always straightforward, since the use of many different data types could be necessary. A common example is a mix of Boolean and numeric functions: function sets could exist composed of Boolean functions (*AND*, *OR*, ...), arithmetic functions (+, −,

$*$, $/$, ...), comparison functions ($>$, $<$, $=$, ...), conditionals ($IF\ THEN\ ELSE$), etc. and one might want to evolve expressions such as:

$$IF\ ((x > 10 * y)\ AND\ (y > 0))\ THEN\ z + y\ ELSE\ z * x$$

In such cases, introducing typed functions into the GP genome can help to force the closure property to be verified. GP in which each node carries its type as well as the types it can call, thus forcing functions calling it to cast the argument into the appropriate type, is called *strongly typed GP* [Banzhaf et al., 1998]. Using types might make even more sense in GP than with a human programmer, since a human programmer has a mental model of what needs to be done, whereas the GP system is completely random in its initialization and variation phases. Furthermore, type checking reduces the search space, which is likely to facilitate the search.

The sufficiency property requires that the set of terminals and the set of functions be capable of expressing a solution to the problem. For instance, the function and terminal sets of the previous example verify the sufficiency property if the problem at hand is an arithmetic one. It does not verify this property, for instance, if the problem faced is a logic one. For many domains, the requirements for sufficiency in the function and terminal sets are not clear and the definition of appropriate sets depends on the experience and the knowledge of the problem of the GP designer.

8.1.2 Initialization of a GP Population

Initialization of the population is the first step of the evolution process. It consists in the creation of the program structures that will later be evolved. The most common initialization methods in tree-based GP are the *grow* method, the *full* method and the *ramped half-and-half* method [Koza, 1992]. These methods will be explained in the following paragraphs, where the set of function symbols composing the trees will be denoted by $\mathscr{F}$, the set of terminal symbols by $\mathscr{T}$ and the maximum depth allowed for the trees by d.

Grow initialization. When the grow method is employed, each tree of the initial population is built using the following algorithm:

- a random symbol is selected with uniform probability from $\mathscr{F}$ to be the root of the tree;
- let n be the arity of the selected function symbol. Then n nodes are selected with uniform probability from the set $\mathscr{F} \cup \mathscr{T}$ to be its sons;
- for each function symbol between these n nodes, the method is recursively applied, i.e., its sons are selected from the set $\mathscr{F} \cup \mathscr{T}$, unless this symbol has a depth equal to $d-1$. In the latter case, its sons are selected from $\mathscr{T}$.

In other words, the root is selected with uniform probability from $\mathscr{F}$, so that no tree composed of a single node is created initially (even though, in some implementations, trees composed of one single node can be admitted in the initial population).

Nodes with depths between 1 and $d-1$ are selected with uniform probability from $\mathscr{F} \cup \mathscr{T}$, but once a branch contains a terminal node, that branch has ended, even if the maximum depth d has not been reached. Finally, nodes at depth d are chosen with uniform probability from $\mathscr{T}$. Since the incidence of choosing terminals from $\mathscr{F} \cup \mathscr{T}$ is random throughout initialization, trees initialized using the grow method are likely to have irregular shape, i.e., to contain branches of various different lengths.

Full initialization. Full initialization works like grow initialization, with the difference that, instead of selecting nodes from $\mathscr{F} \cup \mathscr{T}$, the full method chooses only function symbols (i.e., symbols in $\mathscr{F}$) until a node is at a depth equal to $d-1$. Then it chooses only terminals (i.e., symbols in $\mathscr{T}$). The result is that every branch of the tree goes to the full maximum depth.

Ramped half-and-half initialization. As first noted by Koza [Koza, 1992], a population initialized with the above two methods may be composed of trees that are too similar to each other. In order to increase population diversity, the ramped half-and-half technique has been developed. Let d be the maximum-depth parameter. The population is divided equally among individuals to be initialized with trees having maximum depths equal to 1, 2, ..., $d-1$, d. For each depth group, half of the trees are initialized with the full technique and half with the grow technique.

8.1.3 Fitness Evaluation

Each program in the population is assigned a fitness value, representing its ability to solve the problem. This value is calculated by means of some well-defined explicit procedure. The two fitness measures most commonly used in GP are raw fitness and standardized fitness. They are described below.

Raw Fitness. Raw fitness, as defined by Koza, is "the measurement of fitness that is stated in the natural terminology of the problem itself". In other words, raw fitness is the most simple and natural way to calculate the ability of a program to solve a problem. For example, if the problem consists in driving a robot to make it pick up the maximum possible number of objects contained in a room, the raw fitness of a program that drives the robot could be the number of objects effectively picked up after its execution.

Often, but not always, raw fitness is calculated over a set of *fitness cases*. A fitness case corresponds to a representative situation in which the ability of a program to solve a problem can be evaluated. For example, consider the problem of generating an arithmetic expression that approximates a polynomial, like for instance $x^4+x^3+x^2+x$, over the set of natural numbers smaller than 10. Then, a fitness case is one of those natural numbers. Suppose x^2+1 to be one expression to evaluate, then we say that $2^2+1=5$ is the value assumed by this expression over the fitness case

2. Raw fitness is then defined as the sum of the distances over all fitness cases between values returned by perfect solutions and values returned by individuals to be evaluated. Formally, the raw fitness f_R of an individual i, calculated over a set of N fitness cases, can be defined as:

$$f_R(i) = \sum_{j=1}^{N} |S(i,j) - C(j)|^k$$

where $S(i,j)$ is the value returned by the evaluation of individual i over fitness case j, $C(j)$ is the correct value for fitness case j and $k \in \mathbb{N}$.

Fitness cases are typically a small sample of the entire domain space and they form the basis for generalizing the results obtained to the entire domain space. The choice of how many fitness cases to use, and which ones, is often crucial and it may depend on the available data, on the experience of the GP designer and her knowledge of the problem.

Standardized Fitness. Standardized fitness restates the raw fitness so that a lower numerical value is always a better value. For cases in which lesser values of raw fitness are better, it can happen that standardized fitness equals raw fitness. In many cases, it is convenient and desirable to make the best value of standardized fitness equal zero. If not already the case, this can be achieved by subtracting or adding a constant. If, for a particular problem, a greater value of raw fitness is better, and the maximum possible value of raw fitness f_R^{max} is known, standardized fitness f_S of an individual i can be defined as:

$$f_S(i) = f_R^{max} - f_R(i)$$

where $f_R(i)$ is the raw fitness of i.

8.1.4 Selection

As pointed out in Chapter 3, selection depends on the phenotype of the individuals, while the genetic operators depend on their genotype. Given that what makes a difference between GP and GAs is the genotype, i.e., the structures representing the individuals evolving in the population, the selection operators of GP are identical to those of GAs. In particular, GP can be implemented using fitness proportionate selection (roulette wheel), ranking selection and tournament selection. The interested reader is referred to Section 3.1 for a presentation of these selection algorithms. In GP, tournament selection is the most common choice. Numerous other selection algorithms have been developed for GP, such as tournaments that select parents based on more than one criterion [Luke and Panait, 2002] (which allows the implementation of pseudo-multiobjective evolution), and lexicase selection [Helmuth et al., 2015] (which maintains higher levels of population diversity, a desirable property for solving most problems), just to name a few.

8.1.5 Genetic Operators

Given that GP differs from GAs in the genotype of the individuals evolving in the population, new genetic operators of crossover and mutation have to be defined for GP. The standard GP crossover and mutation are presented in the continuation.

Crossover. The crossover (sexual recombination) operator creates variation in the population by producing new offspring that consist of parts taken from each parent. The two parents, which will be called T_1 and T_2, are chosen by means of a selection algorithm. Standard GP crossover [Koza, 1992] begins by independently selecting one random point in each parent (it will be called the crossover point for that parent). The crossover fragment for a particular parent is the subtree rooted at the node lying underneath the crossover point. The first offspring is produced by deleting the crossover fragment of T_1 from T_1 and inserting the crossover fragment of T_2 at the crossover point of T_1. The second offspring is produced in a symmetric manner. Figure 8.2 shows an example of standard GP crossover.

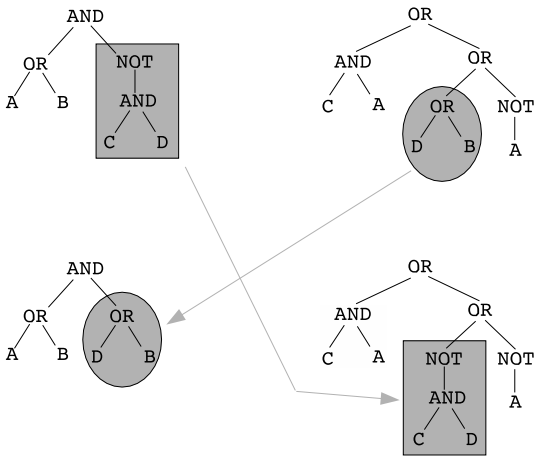

Fig. 8.2 An example of standard GP crossover. Crossover fragments are marked by gray shapes

Because entire subtrees are swapped and because of the closure property of the functions, crossover always produces syntactically legal programs, regardless of the selection of parents or crossover points.

It is important to remark that in cases where a terminal and/or the root of one parent are located at the crossover point, generated offspring could have considerable depths. This may be one possible cause of the phenomenon of *bloat*, i.e., progressive growth of the code size of individuals in the population, which will be discussed later. For this reason, many variants of the standard GP crossover have been proposed in the literature. The most common ones consist in assigning different probabilities of being chosen as crossover points to the various parents' nodes,

depending on the depth level at which they are situated. In particular, it is very common to assign low probability of being selected as crossover points to the root and the leaves, so as to limit the influence of degenerative phenomena like the ones described above. Another different kind of GP crossover is *one-point crossover*, introduced in [Poli and Langdon, 1998a]. It deserves to be mentioned for the importance it has had in the development of a solid GP theory (see Section 8.2).

Mutation. Mutation is asexual, i.e., it operates on only one parental program. Standard GP mutation, often called *subtree* mutation, begins by choosing a point at random, with uniform probability distribution, within the selected individual. This point is called the mutation point. Then, the subtree laying below the mutation point is removed and a new randomly generated subtree is inserted at that point. Figure 8.3 shows an example of standard GP mutation.

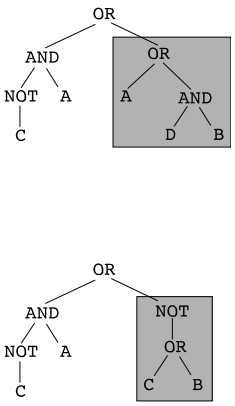

Fig. 8.3 An example of standard GP mutation

This operation, as is the case for standard crossover, is controlled by a parameter that specifies the maximum depth allowed and limits the size of the newly created subtree that is to be inserted. Nevertheless, the depth of the generated offspring can be considerably larger than that of the parent.

As for crossover, many variants of standard GP mutation have been developed too. The most commonly used are aimed at limiting the probability of selecting the root and/or the leaves of the parent as mutation points. A special mention is deserved by *point mutation* [Poli and Langdon, 1997], which exchanges a single node with a random node of the same arity, for the importance it has had in GP theory (see Section 8.2). Other commonly used variants of GP mutation are: *permutation* (or *swap* mutation), which exchanges two arguments of a node, and *shrink mutation*, which generates a new individual from a parent's subtree.

8.1.6 Survival

Once the offspring are created, the new population may completely replace the previous one, or there may be a selection of which individuals survive, among the old and new populations. Like in GAs, also in GP it is common to use some form of *elitism*, in which the best k individuals among old and new are copied unchanged into the next population, guaranteeing their survival. This parameter k is the size of the elite, and to turn off elitism one simply has to set $k = 0$. Again, like in GAs, the genetic operator of replication, described above, also represents a soft form of elitism, as it also (probabilistically) promotes the survival of the best individuals into the next population.

8.1.7 GP Parameters

Once the GP user has decided the set of functions $\mathscr{F}$ and the set of terminals $\mathscr{T}$ used to represent potential solutions of a given problem, and once the exact implementation of the genetic operators to employ has been chosen, still some parameters that characterize evolution need to be set. A list comprising some of these parameters is the following one:

- Population size.
- Population initialization algorithm.
- Selection algorithm.
- Crossover type and rate.
- Mutation type and rate.
- Maximum tree depth.
- Elitism.
- Stopping criteria.

Some of these choices arrive with more parameters to set, for example, tournament selection requires choosing the tournament size, and elitism requires choosing the elite size and the replication rate, the same way it did in GAs. The importance of parameter setting in EAs is a controversial issue. For instance, some work suggests that EAs are mostly insensitive to parameter choices [Sipper et al., 2018], while others seem to have a different opinion, at least for some versions of GP [Trujillo et al., 2020]. In general, much of what GP researchers know about parameter setting is empirical and based on experience.

Besides these parameters, an array of different implementation choices may result in wildly different evolutionary dynamics and capabilities. Like in GAs, also in GP we may have a steady-state instead of generational implementation, or a multi-objective approach, just to name a few. Specific to GP, we may have different bloat control methods, of which the maximum tree depth is one of the simplest, and we may have Automatically Defined Functions (ADFs) or other modular structures, just to name a few. The usefulness of ADFs has been shown by Koza in [Koza, 1994],

later followed by other modular constructs [Koza et al., 1999], all aimed at allowing a high degree of code reusability inside GP individuals.

8.1.8 An Example Run

A very simple example GP run is discussed here. The problem to be solved is the even parity 2 problem, and it consists in finding a Boolean function of two Boolean arguments that returns *true* if an even number of its arguments are true and *false* otherwise (a generalization called the even parity k problem will be defined later in this chapter). Let A and B be the names of the two arguments; a Boolean function $f(A,B)$ perfectly solving this problem must respect the truth table in Table 8.1. Every line of

A	B	$f(A,B)$
false	false	true
false	true	false
true	false	false
true	true	true

Table 8.1 Truth table of the optimal individual for the even parity 2 problem

this table represents a fitness case. For every Boolean function of arguments A and B, its raw fitness is defined as the number of hits over all the fitness cases. Since the maximum raw fitness value, in this case, is equal to 4, we define the standardized fitness as equal to 4 minus the raw fitness. In this way, an optimal solution has a standardized fitness equal to 0 and the worst individuals have a standardized fitness equal to 4.

Let the following set of GP parameters be used: population size of 4 individuals, tournament selection of size 2, crossover rate equal to 50%, mutation rate equal to 25% and replication rate equal to 25%, maximum tree depth equal to 4 and ramped half-and-half initialization.[4] Let individuals be built with the set of functions $\mathscr{F} = \{and, or, not\}$ and the set of terminals $\mathscr{T} = \{A, B\}$.

The process begins with the generation of the initial population by the ramped half-and-half technique. Since the maximum tree depth is 4 and the population size is 4, the initial population will probably be composed of one individual of depth 1, one individual of depth 2, one individual of depth 3 and one individual of depth 4. Let the individuals composing the initial population be the following ones (prefix expressions instead of tree representations are given for the sake of simplicity):

1. $T_1 = not(A)$
2. $T_2 = or(and(A,B), and(A,A))$
3. $T_3 = and(and(A,B), or(A,A))$

[4] The reader is warned that these parameters are unrealistic and have been used for the sake of simplicity, just to show a first example of a GP run.

4. $T_4 = not(or(B, not(or(A,B))))$

The following steps consist in evaluating the fitness of all the individuals composing the population. Truth tables of individuals T_1, T_2, T_3, T_4 are shown in Figure 8.4, tables (a) through (d), respectively. Let f_R be the raw fitness and f_S be the standardized

A	B	T_1
false	false	true
false	true	true
true	false	false
true	true	false

(a)

A	B	T_2
false	false	false
false	true	false
true	false	true
true	true	true

(b)

A	B	T_3
false	false	false
false	true	false
true	false	false
true	true	true

(c)

A	B	T_4
false	false	false
false	true	false
true	false	true
true	true	false

(d)

Fig. 8.4 (a) Truth table of individual $not(A)$; (b) Truth table of individual $or(and(A,B), and(A,A))$; (c) Truth table of individual $and(and(A,B), or(A,A))$; (d) Truth table of individual $not(or(B, not(or(A,B))))$

fitness. From these tables, the following fitness values can be calculated:

1. $f_R(T_1) = 2 \rightarrow f_S(T_1) = 2$
2. $f_R(T_2) = 2 \rightarrow f_S(T_2) = 2$
3. $f_R(T_3) = 3 \rightarrow f_S(T_3) = 1$
4. $f_R(T_4) = 1 \rightarrow f_S(T_4) = 3$

Given the rates of replication, crossover and mutation, one individual will probably be selected for replication, two individuals for crossover and one for mutation. Let the tournament selection be first applied between T_1 and T_4. T_1 has a better fitness value and thus T_1 is reproduced, i.e., copied as it is into the new population. Analogously, let the two individuals chosen with the tournament technique for crossover be T_2 and T_3 and let

$$T_5 = and(and(A,B), and(A,A)), \quad T_6 = or(and(A,B), or(A,A))$$

be the two offspring to be inserted into the new population. Finally, let T_2 be the individual selected for mutation and let:

$$T_7 = or(and(A,B), and(A, not(B)))$$

be the generated offspring. Then, the new population is:

1. $T_1 = not(A)$

2. $T_5 = and(and(A,B),and(A,A))$
3. $T_6 = or(and(A,B),or(A,A))$
4. $T_7 = or(and(A,B),and(A,not(B)))$

At this point, the process is iterated on this new population. Truth tables analogous to the ones of Figure 8.4 could be written for T_1, T_5, T_6 and T_7. These tables would allow one to calculate the following values of standardized fitness:

1. $f_S(T_1) = 2$
2. $f_S(T_5) = 1$
3. $f_S(T_6) = 2$
4. $f_S(T_7) = 2$

Let T_6 be the individual selected for replication. Let T_1 and T_7 be the individuals selected for crossover and let

$$T_8 = A, \quad T_9 = or(and(A,B),and(not(A),not(B)))$$

be the offspring to be inserted into the new population. Finally, let T_5 be the individual selected for mutation and let

$$T_{10} = and(and(A,B),B)$$

be the generated offspring. Then, the new population is:

1. $T_6 = or(and(A,B),or(A,A))$
2. $T_8 = A$
3. $T_9 = or(and(A,B),and(not(A),not(B)))$
4. $T_{10} = and(and(A,B),B)$

Iterating the process, and thus evaluating the fitness of all the individuals in the new population, T_9 is discovered to have a standardized fitness equal to 0, and thus it is an optimal solution. This leads the process to stop and to return T_9 as a solution to the given problem.

An important remark can be made on this example: consider, for instance, individual T_6: $or(and(A,B),or(A,A))$. This expression is clearly equivalent to $or(and(A,B),A)$ (in the sense that these two expressions have identical truth values). From this consideration, it is straightforward to deduce that GP individuals are not necessarily (and in general *are not*) in their most natural form. If the final user needs to have solutions in such a form, a *simplification* phase is necessary. The steps that enable simplification of the structure of solutions are often called *rewrite steps*. This phase clearly depends on the type of language used to code GP individuals and it cannot be generalized.